

GPU Acceleration

What runs on the device, what it costs, what it was measured at, and the bug the verification found

2 September 2026

Contents

1. Why this component exists at all	2
2. What is on the device	2
3. What it was measured at	3
3.1 Speedup on the solve	3
3.2 The device computes the same thing, not something close	3
3.3 Where the time actually goes	4
3.4 Presolve on the device	5
4. The bug the verification found	5
4.1 And the reason it nearly went unnoticed	6
5. Honest limits	6
6. Reproducing this	6

1. Why this component exists at all

The problem statement asks for an **indigenous, GPU-accelerated optimization solver**. The word “GPU-accelerated” is not decoration in that sentence — it constrains the choice of algorithm far more than it constrains the code, and the constraint runs backwards from the hardware to the mathematics.

Three families of algorithm solve a linear program.

	mechanism	can a GPU help?
Simplex	walk vertex to vertex along edges	no — inherently sequential
Interior point	Newton steps through the interior	barely — the cost is a sparse factorisation
First-order	gradient step, then projection	yes

The reason is visible in the first-order iteration itself:

$$x^{k+1} = \Pi_{[l,u]}(x^k - \tau(c - K^\top y^k)), \quad y^{k+1} = \Pi_{y \geq 0}(y^k + \sigma(q - K\bar{x}))$$

Two sparse matrix-vector products, some vector arithmetic, two clamps. No basis, no factorisation, nothing sequential. A sparse product is thousands of independent row dot-products; a clamp is elementwise. **This is the only LP algorithm whose inner loop is natively parallel**, and it is why every GPU LP solver in the literature — PDLP, cuPDLP, cuPDLPx, HPR-LP — is in this family.

So the GPU did not get bolted onto an existing solver. The algorithm was chosen because the hardware requirement demanded it.

2. What is on the device

Everything in the inner loop, and — since the last round of work — the convergence machinery as well.

kernel	what it does
<code>spmv K x, spmv Kt y</code>	the two sparse products
<code>primal update</code>	gradient step, projection, reflection and Halpern blend, fused
<code>dual update</code>	the dual half, plus K applied to the point it produces
<code>convergence terms</code>	the residuals and the gap, computed on the device
<code>weighted norm, reduce sum, scale into step-size copy back, convergence copy back</code>	the reductions the only host transfers left

Two design decisions are worth naming because they are where the performance is.

The interface is deliberately coarse. It does not expose “add two vectors”; it exposes `primal_update` and `dual_update`, which are whole steps of the algorithm. A fine-grained interface would mean one kernel launch and one full pass over memory per elementary operation. Fusing the projection, the reflection and the Halpern blend into a single kernel turns five sweeps into one and five launches into one.

And one sparse product per iteration is avoided by algebra rather than by code. Since $K\bar{x} = 2Kx_{\text{pdhg}} - Kx$ and K is linear, the reflection and the Halpern blend can be applied to Kz as well as to z . Recomputing Kx instead costs a whole sparse product every iteration — which on this instance set was the entire difference between Halpern winning and losing.

Adaptive SpMV width. The number of lanes assigned per row is chosen from the average row length rather than fixed at a warp, so a matrix of short rows does not waste 28 of every 32 lanes. `spmv K x` went from 0.207 s to 0.042 s on the instance that motivated it.

3. What it was measured at

Hardware: NVIDIA Tesla T4, 15,360 MiB, CUDA 12.8, on Kaggle. **Verified:** 2 September 2026. All fourteen test suites pass on the device, including the backend contract tests.

3.1 Speedup on the solve

instance	CPU solve	GPU solve	speedup	CPU/GPU agreement
supportcase10	88.23 s	7.31 s	12.07×	1.3e-16
gap15	0.86 s	0.15 s	5.68×	8.7e-16
datt256_lp	4.93 s	1.06 s	4.64×	5.6e-16
graph40-40	2.29 s	0.73 s	3.14×	0.0e+00

Agreement is the CPU answer against the GPU answer: at or below machine precision on all four, and **exactly zero** on one.

Setup is excluded, and it is identical on both sides. It is MPS parsing and standard-form construction, which run on the host and have nothing to do with the backend. Including it would dilute the comparison by a constant — Amdahl’s law quietly eating the result. End-to-end wall clock is recorded in `results/gpu_matrix.csv` for anyone who wants it.

These numbers replace an earlier **2.70×**–**7.09×**, measured before the device-side convergence check, the fused updates and the adaptive SpMV width.

3.2 The device computes the same thing, not something close

Fourteen Netlib instances solved on both backends against published optima: **zero instances off the published optimum**, and CPU and GPU take the **same number of iterations on every one**.

instance	CPU iterations	GPU iterations
25fv47	45,040	45,040
share1b	36,320	36,320
bandm	27,520	27,520
fit1p	25,360	25,360

Identical iteration counts are a stronger statement than matching objectives. Two implementations can reach the same answer by different paths; taking the same number of steps says the device is running the algorithm rather than an approximation of it.

3.3 Where the time actually goes

The backend carries its own per-kernel timing, so this needs no external profiler — which matters, because `nsys` is not on the Kaggle image.

qap15, 2,600 iterations — the case where a GPU pays:

	share
iterating	79.5%
checks	5.4%
polishing	8.8%
scaling + norm + setup	6.4%
off the device	11.8%

and inside the kernels:

kernel	share	launches
<code>spmv Kt y</code>	36.2%	2,767
<code>spmv K x</code>	30.3%	2,711
<code>primal update</code>	17.6%	2,680
<code>dual update</code>	12.5%	2,680
everything else	3.4%	308

Two thirds in the two sparse products is the shape this algorithm should have.

graph40-40, 200 iterations — the opposite case, kept deliberately:

	share
polishing	43.9%
scaling	25.5%
iterating	20.7%
off the device	35.4%

An instance that converges in 200 iterations cannot be made fast by a better kernel, because the kernels are not where its time goes. That is a real limit of GPU acceleration for this method and it is worth showing rather than asserting: **the GPU pays in proportion to how many iterations the instance needs.**

This also explains the spread in §3.1. `supportcase10` runs 37,760 iterations and gets 12.07×. `graph40-40` runs 200 and gets 3.14×.

3.4 Presolve on the device

instance	backend	presolve	iterations	seconds
supportcase10	cpu	off	37,760	88.49
supportcase10	gpu	off	37,760	8.50
supportcase10	gpu	on	37,840	7.91
qap15	cpu	off	2,600	0.89
qap15	gpu	off	2,600	0.78
datt256_lp	cpu	off	440	6.25
datt256_lp	gpu	off	440	3.05

Presolve and the GPU are independent wins and they compose — but note `qap15`, where presolve *increases* the iteration count from 2,600 to 4,320 while still being faster in wall clock on the GPU. Presolve is not uniformly good for a first-order method, and §5 has the cases where it is actively bad.

4. The bug the verification found

This is the part of the report that matters most, and it is the reason the run was worth doing rather than assuming.

The run before this one failed, with 2,344 wrong comparisons in the backend contract test.

`prepare()` uploads a matrix to the device and caches the result so a matrix is not re-uploaded every iteration. It keyed that cache on the **host address**:

```
void prepare(const SparseMatrix& a) const override {
    if (matrices_.count(&a)) return;    // already on the device?
    ...
}
```

A stack-local matrix reuses the address of the one before it the moment that one goes out of scope. So the second matrix silently received the first one's rows, its nonzero count, its vector width and its values. **No error, no warning, no exception — just the wrong answer.**

Why nothing caught it earlier. Nothing inside a solve reuses an address: a matrix and its transpose are constructed once and live for the whole run. That is why the fourteen instances in §3.2 pass on the device with objectives *and iteration counts* identical to the CPU while the backend was, in general, wrong. It took a contract test that builds five matrices in a loop — a regime no real instance reaches — to expose it.

The fix. Identity now comes from the object rather than from where it happens to live: `SparseMatrix` carries an id assigned at construction, and the cache is keyed on that. The regression test builds two different matrices, with different nonzero counts, at the same address — and prints a note if they happen *not* to share an address, so it cannot pass quietly without having exercised the thing it is for.

4.1 And the reason it nearly went unnoticed

The verification harness reported “GPU backend is correct” while the gating test was red. The line responsible:

```
if ctest --test-dir "$build" --output-on-failure 2>&1 | tail -6; then ;; else fail=1; fi
```

A pipeline’s exit status is the exit status of its **last** command. `tail` always succeeds, so `ctest` failing was discarded entirely. The check ran, the failure was real, and nothing carried it to the summary.

A check that cannot fail is not a check, and this one had been in the harness the whole time. It now writes to a file, tests `ctest` itself, and prints forty lines rather than six when it fails — six lines of a failure is not enough to act on, which is why the first diagnosis needed a second manual run.

5. Honest limits

The GPU pays in proportion to iterations. §3.3 shows this directly: `supportcase10` at 37,760 iterations gets 12.07×, `graph40-40` at 200 gets 3.14×. An instance the method solves quickly is an instance the GPU cannot help much with, because its time is in scaling and polishing on the host.

Not at cuPDLP-C level. The literature’s own comparison ([arXiv:2509.23903](https://arxiv.org/abs/2509.23903)) measures HPR-LP against cuPDLPx at 1.1×–1.8× and concludes the difference is *implementation, not algorithm*. This implementation is further back than either, and the remaining distance is engineering rather than a missing idea.

Presolve can hurt the first-order method. Over the full Netlib set with presolve on, three instances **stop converging** that converged without it — `agg2` and `agg3` go from 6,400 and 16,960 iterations to the 1,000,000 cap, and `bore3d` from 244,480 to the cap. The geometric mean is still a 1.38× speedup over 81 instances, best 20.41× and worst 0.01×, but that spread is real and the worst case is a solver that no longer finishes.

One T4, one run. These numbers are from a single rented GPU on a shared Kaggle instance. They are reproducible — `scripts/gpu_test.sh` runs the whole sequence — but they are not a statistically careful benchmark across hardware.

6. Reproducing this

```
bash scripts/package_for_kaggle.sh      # source only, ~1 MB
```

Then on a CUDA box:

```
python3 scripts/fetch_lptestset.py --max=4    # the large instances
bash scripts/gpu_test.sh
```

Eight steps: device check, build, **backend contract tests (the gating check)**, correctness against published optima, CPU-against-GPU by size, presolve interaction, the per-kernel profile, and presolve end to end.

`scripts/check_cuda_syntax.sh` type-checks the kernels with no GPU present, which is what the development machine — Apple Silicon, no CUDA — can do locally.

`docs/KAGGLE.md` has the full workflow.